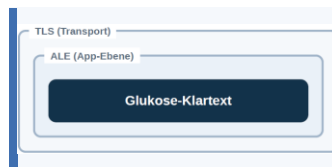


A Wie die Verschlüsselung funktioniert

Ein kompakter Einstieg in die drei Bausteine hinter diesem Bericht, damit die folgende Analyse auch ohne Kryptovorwissen lesbar ist. Bewusst konzeptionell gehalten.

0.1 Ein Tresor im Tresor

Die Vertraulichkeit ruht auf **zwei** voneinander unabhängigen Schichten. Außen liegt das gewohnte **TLS** (das „https“ der Transportverschlüsselung). Innerhalb von TLS liegt der eigentliche Inhalt **noch einmal** verschlüsselt in einer anwendungseigenen Schicht, die man **Application-Level Encryption (ALE)** nennt. Wer nur die äußere Schicht öffnet, findet innen weiterhin einen verschlossenen Kasten und der Klartext entsteht erst *innerhalb* der App.



Folge: Ein klassisches Mitschneiden des Netzwerkverkehrs (TLS-MITM) liefert nur den inneren, verschlüsselten Container — nicht den Messwert.

0.2 Zwei Verfahren: Geheimhaltung und Echtheit

0.2.1 AES-GCM — verschlüsseln (Vertraulichkeit *und* Integrität)

Ein **symmetrisches** Verfahren: Derselbe Schlüssel ver- und entschlüsselt. „GCM“ liefert neben dem Ciphertext ein 16 Byte langes **Siegel** (Auth-Tag). Beim Entschlüsseln wird das Siegel nachgerechnet. Passt es nicht exakt zu Schlüssel, Nonce und Daten, **bricht die Operation ab und liefert nichts**. Manipulation ist damit ausgeschlossen. Eine kurze Einmal-Zahl (die **Nonce**, 12 Byte) sorgt dafür, dass dieselbe Nachricht nie zweimal gleich aussieht.



Entschlüsseln ist der umgekehrte Weg. Stimmt das Tag nicht, gibt es keinen Klartext.

0.2.2 RSA-PSS — signieren (Echtheit)

Ein **asymmetrisches** Verfahren mit Schlüsselpaar: Der **private** Schlüssel *signiert*, der **öffentliche** *prüft*. Signiert wird nicht die ganze Nachricht, sondern ihr Hash plus einem Zufalls-Salt („PSS“). Dadurch sieht jede Signatur anders aus, obwohl sie dieselbe Nachricht bestätigt. RSA-PSS **verschlüsselt nichts**. Es beweist nur, dass eine Nachricht echt und unverändert ist. AES sorgt für Geheimhaltung, RSA für Authentizität. *Woher die öffentlichen und privaten Schlüssel dafür kommen, klärt der Bericht (§7).*

0.3 Der Tresor drumherum: White-Box-Kryptografie

Eine **White-Box** (Fachname „Secure Key Box“) rechnet mit Schlüsseln, ohne sie je als lesbaren Wert offenzulegen. Der Schlüssel ist in die Rechenlogik „eingebacken“. Das ist die zentrale Hürde: Man kann ihn nicht einfach aus dem Speicher fischen, weil er dort nie als zusammenhängender Wert auftaucht.

0.4 Und die Schlüssel selbst?

Wie aus diesen Bausteinen die eigentlichen Sitzungsschlüssel entstehen und warum das anders funktioniert, als man zunächst vermutet, ist die **zentrale Erkenntnis** des Berichts. Sie wird Schritt für Schritt hergeleitet (§7) und hier bewusst nicht vorweggenommen. Mit diesen Grundbegriffen im Gepäck lohnt sich der Blick in die Details: Wie sich die Verschlüsselung trotz eines aggressiven Manipulationsschutzes analysieren ließ und wie sich die Kette am Ende sogar in einem eigenen Prozess vollständig reproduzieren ließ.

B LibreLinkUp Application-Level-Encryption ein Reverse-Engineering-Writeup

Technischer Erfahrungsbericht über die Krypto- und Schutz-Architektur der LibreLinkUp-Android-App und die Wege, den **eigenen** Glukosewert für die **eigene** Smartwatch nutzbar zu machen.

Autor: lamDiesel · **Stand:** September 2026 · **Version:** 1.0 · **Lizenz:** [CC BY 4.0](#)

Status: Ziel erreicht (zwei unabhängige Verfahren, eines davon offline verifiziert). Durchgehend auf einem **ungerooteten** Gerät. **Fokus:** Verständnis & Methodik. Diese öffentliche Fassung ist bewusst ein *Erkenntnisbericht*, kein Kochrezept.

⚠ **Medizinischer Haftungsausschluss.** Dies ist ein technischer Sicherheits- und Reverse-Engineering-Bericht, **kein medizinisches Dokument**. Die beschriebenen Verfahren und sämtliche damit gewonnenen Glukosewerte (CGM-Daten) dürfen **unter keinen Umständen** für medizinische Entscheidungen herangezogen werden — insbesondere nicht für die Dosierung von Insulin oder andere Therapieanpassungen. Selbst erstellte Auslese-Wege können fehlerhafte, verzögerte, veraltete oder unvollständige Werte liefern. Für Therapieentscheidungen sind ausschließlich die vom Hersteller zugelassenen, geprüften Anzeigegeräte und -Apps sowie ärztlicher Rat maßgeblich.

Scope, Ethik & bewusste Auslassungen

Das beschriebene Vorgehen betraf ausschließlich die **eigene** App-Installation, die **eigenen** Zugangsdaten und die **eigenen** Gesundheitsdaten auf einem Gerät im eigenen Besitz. Motivation war persönliche Interoperabilität: Den eigenen Wert auf der eigenen Uhr sehen.

Wichtig war mir dabei, das gesamte Vorgehen auf einem **ungerooteten** Gerät im Alltagszustand durchzuführen. Ohne Root, ohne Custom-ROM, ohne Aushebeln der Plattform-Sicherheit. Das hält das Gerät in seinem normalen, vertrauenswürdigen Zustand (kein Garantieverlust, keine geschwächte Android-Sicherheit) und entspricht dem realistischen Rahmen eines gewöhnlichen Nutzers. Zugleich ist es die **höhere Hürde**: Viele gängige Hook- und Instrumentierungs-Ansätze setzen Root voraus, der hier beschriebene Weg kommt bewusst ohne aus.

Diese Veröffentlichung ist als **Security-/RE-Research-Writeup** gedacht. Sie erklärt Architektur, Konzepte und Methodik, damit andere das *Verständnis* nachvollziehen können. Sie enthält **absichtlich nicht**:

- lauffähige Exploit-/Hook-Quelltexte,
- die konkreten Smali-Patches, die den Manipulationsschutz der App neutralisieren,
- build-spezifische Speicheradressen (Offsets) und Krypto-Konstanten und
- Schlüsselmaterial oder personenbezogene Daten jeglicher Art.

Diese Bausteine würden den Text in ein schlüsselfertiges Werkzeug zur Umgehung des Schutzes einer fremden App verwandeln, das ist nicht das Ziel. Wer denselben Weg für seine eigenen Daten geht, muss die gerätespezifischen Details selbst erarbeiten. Genau das ist Teil der Legitimitäts-Schwelle.

1. Das Problem in einem Satz

LibreLinkUp verschlüsselt seine API-Nutzlast **doppelt**: Einmal per TLS (der normale HTTPS-Transport) und darüber hinaus noch einmal **innerhalb der App** (Application-Level Encryption, „ALE“). Selbst wer TLS aufbricht, sieht deshalb nur einen verschlüsselten Container und nicht den Glukosewert.

2. Systemüberblick

Baustein	Rolle
Die App (org.nativescript.LibreLinkUp)	„Follower“-App; App-Logik in JavaScript (NativeScript), nativer Unterbau.
Eine native Krypto-Bridge (in Go geschrieben)	Kapselt die eigentliche Verschlüsselung; enthält eine White-Box-Krypto (Secure Key Box).
Eine native Schutz-Bibliothek (RASP)	Runtime Application Self-Protection. Erkennt Manipulation/Debugger/Root und lässt die App gezielt abstürzen.
Kommerzieller Obfuskator (DexGuard)	Verschleiert den Java/Smali-Code und bringt die Integritätsprüfungen mit.

Kernpunkt: Die Vertraulichkeit hängt **nicht** an TLS, sondern an der ALE-Schicht und die wiederum an der White-Box in der Go-Bridge.

3. Die ALE-Schicht

Jeder verschlüsselte Body hat konzeptionell die Form:

```
{ "data": "<IV>.<Ciphertext>.<Auth-Tag>", // base64, punktgetrennt  
  "signature": "<RSA-Signatur über data>" }
```

- **AES-GCM-128** liefert Vertraulichkeit *und* Integrität (12-Byte-Nonce, 16-Byte-Auth-Tag).
- **RSA-PSS** signiert den data-Teil (Authentizität).

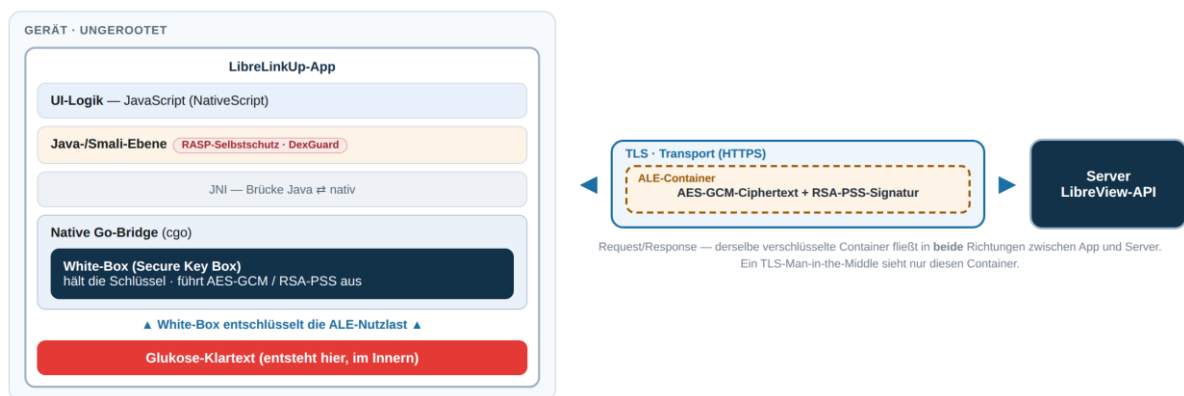
Zur Einordnung: Die **punktgetrennte Zusammensetzung** IV.Ciphertext.Auth-Tag ist eine **app-spezifische Verpackung** von LibreLinkUp. **Kein** kryptografisches Paradigma. AES-GCM erzeugt standardmäßig nur Ciphertext und einen Auth-Tag. Wie diese Bestandteile zusammen mit dem IV/der Nonce serialisiert werden (hier Base64 mit Punkt als Trennzeichen), ist eine reine Design-Entscheidung der App und variiert von Implementierung zu Implementierung.

Ein klassischer TLS-Man-in-the-Middle liefert damit nur diesen Container, der Grund, warum ein On-Device-Ansatz nötig ist.

Die Gesamtarchitektur und wo entschlüsselt wird, fasst Abbildung 1 zusammen:

App-Architektur — und wo entschlüsselt wird

Der Verkehr läuft horizontal zwischen App und Server; die Entschlüsselung passiert erst tief in der App.



Zwei Verschlüsselungen: TLS schützt den Transport, ALE zusätzlich die Nutzlast (AES-GCM + RSA-PSS). · **Trennung der Ebenen:** UI in JavaScript, Selbstschutz auf der Java-/Smali-Ebene, die eigentliche Krypto in der nativen Go-Bridge. · **Entschlüsselung:** nur die White-Box in der Go-Bridge macht aus dem ALE-Container den Klartext.

Abbildung 1: App-Architektur — und wo entschlüsselt wird.

App-Architektur: UI in JavaScript (NativeScript), darunter die Java-/Smali-Ebene mit dem RASP-Selbstschutz. Über JNI die native Go-Bridge mit der White-Box (Secure Key Box). TLS umschließt den ALE-Container (AES-GCM + RSA-PSS). Erst die White-Box in der Go-Bridge entschlüsselt die Nutzlast, sodass der Glukose-Klartext im Innern der App entsteht. Ein TLS-Man-in-the-Middle sieht nur den verschlüsselten ALE-Container.

4. Die Krypto-Bridge existiert erst zur Laufzeit

Die Bridge liegt in der APK nicht als normale, lesbare Bibliothek, sondern **gepackt/verschlüsselt**. Erst beim Start entpackt ein nativer Loader sie in eine **temporäre** Datei im privaten App-Ordner, lädt sie und **löscht sie sofort wieder** (der offene Datei-Deskriptor hält sie am Leben). Konsequenzen:

- Die APK-Kopie ist für die Analyse wertlos. Der echte Code existiert nur im **entschlüsselten Laufzeit-Zustand**.
- Das Zeitfenster ist winzig. Die Datei wächst progressiv und verschwindet dann → Man muss den Schreibvorgang „berennen“, um ein vollständiges Abbild zu bekommen.
- Genau über dieses entschlüsselte Abbild bildet der Manipulationsschutz zur Laufzeit die **Prüfsumme**. An diesem nativen Code darf ein Hook deshalb nichts verändern.

Aus dem vollständigen Abbild lassen sich die exportierten Krypto-Funktionen über die dynamische Symboltabelle des ELF ablesen. Hinweis: Die Debug-Symbole sind entfernt, die *dynamischen* Exporte müssen aber vorhanden bleiben, sonst wäre die Bibliothek nicht nutzbar.

5. Die Schutzschichten (warum es schwierig ist)

- **Isolierter Linker-Namespace**: Standard-Symbolsuche aus einer anderen Bibliothek findet die Funktionen nicht. Adressen müssen über die Modul-Basis aus `/proc/<pid>/maps` bestimmt werden.
- **Native Integritätsprüfung**: Über den Code der Bridge wird zur Laufzeit eine Prüfsumme gebildet. Jeder Byte-Patch an diesem Code fällt auf.
- **Java-Ebene-RASP**: Ist für die Lifecycle-Methoden umgesetzt und ist die wirksamste Schutzschicht: Bei erkannter Manipulation reagiert die App bewusst destabilisierend (zufällige Ausnahmen auf mehreren Threads, eine Warn-Meldung, Speicher-Allokationsbomben) und beendet sich **vor** dem Login.
- **Dynamisch registrierte JNI-Methoden**: Die Methoden der Schutz-Bibliothek sind dynamisch registriert, daher kann man sie nicht ohne Weiteres durch einen Dummy ersetzen.
- **Frida-Erkennung**: Bekannte Instrumentierungs-Signaturen werden im Speicher entdeckt.

Merksatz: Stabil bleibt die App nur, wenn man **beide** Ebenen adressiert. Den nativen Code *nicht* verändern **und** die Java-RASP entschärfen.

Fahrplan der nächsten Kapitel: Zuerst der direkte Weg zum Klartext (**Verfahren 1**). Daraus abgeleitet das **Schlüssel-Modell** und darauf aufbauend der eigenständige Weg ohne laufende App (**Verfahren 2**).

6. Verfahren 1 — am Klartext abgreifen (Live-Hook)

Egal wie gut der Schlüssel geschützt ist: Die Entschlüsselungsfunktion **muss** irgendwann den Klartext in einen Ausgabepuffer schreiben. Wer diesen Puffer nach dem Aufruf liest, hat den Wert, ohne den Schlüssel zu kennen.

Der erste, naive Weg ist es den Funktionsanfang mit einem Sprung zu überschreiben. Er funktionierte zwar, lieferte echten Klartext, wurde aber von der Integritätsprüfung erkannt und führte zum Absturz. Er verändert eben Bytes im geschützten Code.

Der **stabile** Weg nutzt **Hardware-Breakpoints**: Die Debug-Register der CPU lösen aus, *bevor* eine Instruktion läuft und das **ohne ein Byte am geschützten Krypto-Code zu ändern**. Die Prüfsumme über diesen Code bleibt dadurch „sauber“. Weil ein Prozess sich nicht selbst so beobachten darf, übernimmt ein winziger mitgestarteter Beobachter-Prozess das Setzen der Breakpoints und liest nach dem Aufruf den entschlüsselten Puffer aus dem Speicher der App.

Wird die App also gar nicht verändert?

Die App selbst wird sehr wohl verändert, die Beobachter-Bibliothek wird über eine Bytecode-Änderung (Smali-Patch) in die App geladen und weitere Patches entschärfen die Selbstschutz-Prüfungen. Der Punkt der Hardware-Breakpoints ist enger: Nur der **prüfsummen-geschützte native Krypto-Code** bleibt unangetastet. Das ist genau der Teil, den die Integritätsprüfung überwacht.

Erkenntnis: Gegen Laufzeit-Integritätsprüfungen schlägt ein *byte-erhaltender* Beobachtungs-Hook (Hardware-Breakpoint) den klassischen *byte-verändernden* Inline-Hook.

7. Das Schlüssel-Modell (die zentrale Erkenntnis)

Dieses Kapitel baut direkt auf Verfahren 1 auf: Derselbe Hardware-Breakpoint, der dort den Klartext ausliest, kann auch die **Argumente** der Krypto-Aufrufe mitschreiben und genau aus diesen Live-Beobachtungen ergibt sich das folgende Schlüssel-Modell. Es ist zugleich die Grundlage dafür wie in Verfahren 2 ganz ohne die laufende App gearbeitet werden kann.

Das intuitive Modell, „der AES-Schlüssel wird pro Sitzung per Diffie-Hellman (ECDH) neu ausgehandelt“, **stimmt für den laufenden Betrieb nicht**. Im **Regelbetrieb** wird **nichts ausgehandelt**: Die Sitzungsschlüssel ergeben sich deterministisch aus **statischem, in der Bibliothek eingebettetem Material** plus einem **pro-Sitzung vom Server gelieferten Paket** (wie gleich gezeigt).

Wie kommt man überhaupt an das Sitzungs-Paket?

Auch der Weg *dorthin* wurde untersucht. Nicht nur der eigentliche Entschlüsselungs-Schritt. Er läuft in zwei Schritten:

1. Anmeldung: Der Login mit E-Mail und Passwort liefert lediglich ein **Zugangs-Ticket** (technisch ein *JWT*). Es trägt die eigene Konto-/Installations-Identität, aber **kein** Schlüsselmaterial.

2. Provisionierung: Ein anschließender Request holt das **pro-Sitzung-Paket**. Es kommt **verschlüsselt in der Server-Antwort** und ist allein mit dem **statischen, installationsübergreifend gleichen** Grund-Baustein lesbar. Diese eine Anfrage ist mit **AES-GCM** verschlüsselt und mit **RSA-PSS** signiert, dieselben zwei Verfahren aus Teil 0 bzw. §3, hier in Aktion.

Und hier liegt der eigentliche Knackpunkt des Standalone-Wegs: Der Server liefert dieses Paket **nicht auf bloßes Nachfragen**. Der Client muss sich dabei **aktiv als Gerät gegenüber dem Backend anmelden**. Ein passives Nachspielen eines abgehörten Aufrufs genügt nicht, sondern läuft ins Leere (der Server antwortet dann nur mit einer Leer-Reaktion). Genau diese **Eigen-Registrierung** war die nicht offensichtliche Hürde, die den eigenständigen Client überhaupt erst funktionieren lässt. Das konkrete Vorgehen dahinter bleibt hier bewusst außen vor.

Damit ist die zunächst offene Frage beantwortet, woher das Sitzungs-Material stammt: **Aus der Server-Antwort**, nicht aus einem lokalen ECDH-Handshake.

Und im laufenden Betrieb?

Ist die Sitzung einmal provisioniert, inklusive der eben beschriebenen Eigen-Registrierung, wird die Original-App nicht mehr gebraucht: Ab dann kommt ein eigenständiger Client für den laufenden Abruf **allein mit E-Mail und Passwort** aus, sprich ohne On-Device-Mitschnitt. Die eigentliche Hürde liegt also **einmalig** in der Provisionierung, nicht im Regelbetrieb.

Wie ist man nach dem Login authentifiziert?

Das Passwort wandert genau **einmal** über die Leitung: Beim Login. Danach weist sich jeder Aufruf über das **Zugangs-Ticket (JWT)** aus: Der Server hat es selbst signiert und erkennt es an seiner eigenen Signatur wieder. Er weiß so, dass der Aufruf zu diesem Konto gehört, ganz ohne erneutes Passwort.

Davon zu trennen ist die **App-/Geräte-Echtheit** über **RSA-PSS**. Hier lohnt eine genaue Unterscheidung, denn es gibt **zwei Signatur-Situationen zu zwei Zeitpunkten**. Signiert wird ausschließlich die **eine Provisionierungs-Anfrage** und zwar bereits mit dem **statischen Vor-Session-Material**. Also *bevor* überhaupt Sitzungsschlüssel existieren. Diese Signatur dient also als Nachweis „echte App der den Server das Sitzungs-Paket herausgeben lässt“. Die eigentlichen Glukose-Abrufe danach sind schlichte, mit dem Ticket **authentifizierte** Anfragen **ganz ohne eigene Signatur**; nur ihre Nutzlast ist zusätzlich **AES-GCM**-verschlüsselt. Der aus der Schlüssel-Leiter (gleich) ableitbare „Signier-Schlüssel“ wird für das reine Lesen also gar nicht gebraucht.

Und die asymmetrischen (öffentlichen/privaten) Schlüssel?

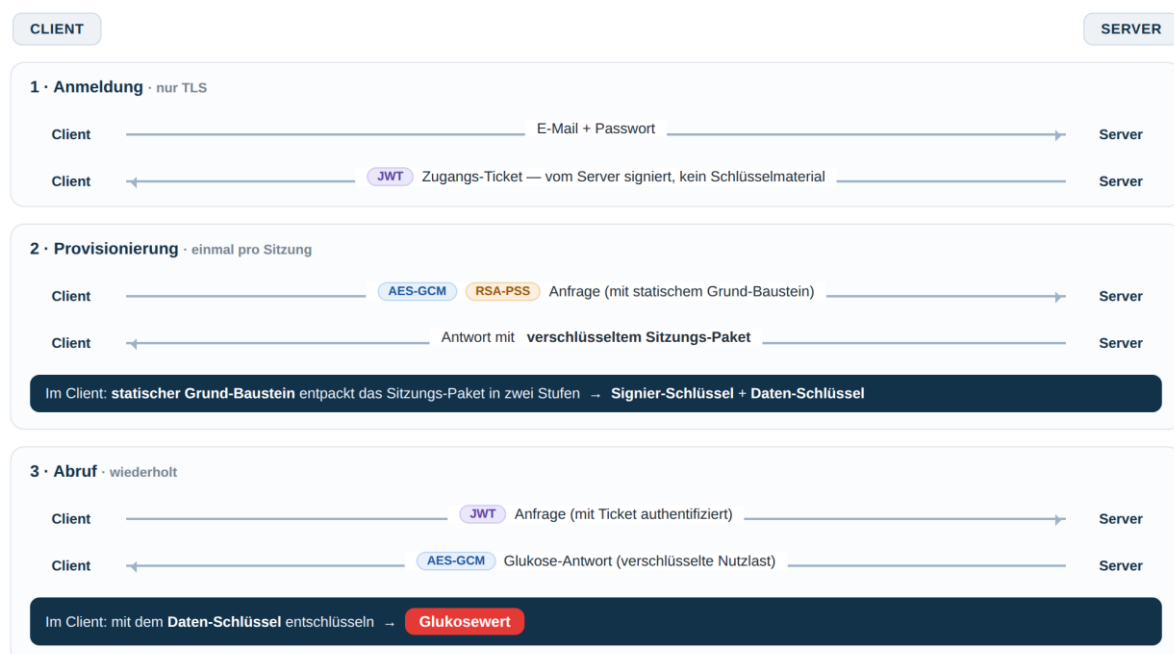
Weil RSA-PSS asymmetrisch ist, stellt sich die Frage nach dem Schlüsselpaar. Für den Lese-/Regelbetrieb ist die Antwort schlang:

- **Signieren (App-Seite):** Der Provisionierungs-Request wird mit dem **statischen, in der Bibliothek eingebetteten** Material signiert. Es wird **kein** frischer privater Schlüssel pro Sitzung erzeugt.
- **Prüfen (Server-Antworten):** Der zum Prüfen nötige **öffentliche Schlüssel des Servers** wird **im Provisionierungs-Paket mitgeliefert**. Ein separater Schlüsselaustausch über einen weiteren Kanal ist nicht nötig.
- **Erst-Einrichtung:** Ein echtes Erzeugen oder Aushandeln eines Geräte-Schlüsselpaars gehört, wenn überhaupt, zur **einmaligen Konto-/Geräte-Einrichtung**. Die ist nicht maßgeblich Gegenstand dieser Untersuchung.

Der folgende Ablauf (Abbildung 2) fasst Login, Provisionierung und Abruf zusammen:

Vom Login zum Glukosewert

Client (eigenes Programm) ↔ Server (LibreView-API) — der Weg von E-Mail + Passwort bis zum entschlüsselten Wert



JWT = signiertes Identitäts-Ticket (Authentifizierung) AES-GCM = Verschlüsselung (Vertraulichkeit + Integrität) RSA-PSS = Signatur (Echtheit)

Abbildung 2: Vom Login zum Glukosewert.

Ablauf: Login → Zugangs-Ticket (JWT), Provisionierung (AES-GCM-verschlüsselt + RSA-PSS-signiert) → verschlüsseltes Sitzungs-Paket → mit statischem Grund-Baustein zu Signier- und Daten-Schlüssel entpacken, dann wiederholter Abruf mit Ticket → AES-GCM-verschlüsselte Glukose-Antwort

Schaut man genauer hin, wie aus dem Sitzungs-Paket ein nutzbarer Schlüssel wird, zeigt sich das eigentliche Modell: Die **zweistufige Schlüssel-Leiter**. Der **statische Grund-Baustein** aus dem Diagramm leistet dabei doppelte Arbeit: Er entschlüsselt bereits die laufenden Daten *und* er **öffnet das Sitzungs-Paket**. Somit startet

er die Leiter. Aus dem Paket werden dann in **zwei Unwrap-Stufen** nacheinander zwei weitere Schlüssel „ausgewickelt“.

Zusammen sind das **drei Rollen**:

Rolle	Herkunft	Wofür
Bootstrap-Schlüssel	der statische, eingebettete Grund-Baustein selbst	Entschlüsselt Konfig-Daten und Request-Bodies, öffnet das Sitzungs-Paket (Start der Leiter) und signiert die Provisionierungs-Anfrage (Vor-Session, s. o.)
Signier-Schlüssel	Stufe 1 des Unwrap (aus dem Sitzungs-Paket)	Steht für Request-Signaturen bereit. Für den reinen Glukose-Abruf aber nicht nötig (die Abrufe tragen nur das Ticket)
Daten-Schlüssel	Stufe 2 des Unwrap (aus dem Sitzungs-Paket)	Entschlüsselt die Nutzdaten u. a. den Glukosewert

Kurz: **Ein** fest eingebauter Baustein startet die Kette und aus dem Server-Paket fallen nacheinander Signier- und Daten-Schlüssel heraus. Für den reinen Glukose-Wert genügt am Ende die **zweite Stufe**. „Der Schlüssel“ ist damit weder ein einzelner statischer Wert noch ein frei ausgehandeltes Geheimnis, sondern das Ergebnis dieses deterministischen Entpackens: **Statischer Baustein + Sitzungs-Paket → Sitzungsschlüssel**.

Woher stammt dieses Modell?

Die Schlüssel-Leiter an sich ist nicht gesetzt, sondern **aus einem Laufzeit-Trace hergeleitet**. Über einen Hardware-Breakpoint auf dem Eintritt der Unwrap- und Entschlüsselungs-Funktionen wurden bei jedem Aufruf die Argumente gemäß der ARM64-Aufrufkonvention ausgelesen. *Als Exkurs AAPCS64: Die ersten Argumente stehen in festgelegten CPU-Registern, weitere auf dem Stack. Man weiß also, in welchem Register welcher Parameter liegt.*

Letztendlich führten drei Beobachtungen zum Modell:

- **Zwei Stufen**: Die Unwrap-Funktion feuerte pro Login **zweimal** mit unterschiedlichem Eingabematerial. Die Ausgabe des ersten Laufs speiste erkennbar den zweiten.
- **Statisch vs. pro-Sitzung**: Die einen Eingabe-Blobs blieben über alle Logins und Geräte **bit-identisch** (sie sind fest in die Bibliothek eingebettet). Die anderen **variieren bei jedem Login** (das sind die server-gelieferten Bundles). Genau dieser Unterschied trennt Bootstrap-Material von Sitzungs-Material.
- **Rollenzuordnung**: Welcher entpackte Schlüssel welche Rolle hat, wurde über die **Objekt-Identität** bestätigt. Es wurde untersucht welcher Schlüssel anschließend welche Operation speist (Signieren vs. Entschlüsseln) und über den **Inhalt des jeweils erzeugten Klartexts**.

Damit ist die Leiter kein axiomatisches Statement, sondern ein am Laufzeitverhalten belegter Befund.

8. Verfahren 2 — die White-Box als Dienstleister (Standalone-Orakel)

Aufbauend auf dem Schlüssel-Modell (§7) lässt sich die Entschlüsselung nun ganz ohne die laufende App nachbauen.

Statt in der laufenden App abzugreifen, kann man die entschlüsselte Krypto-Bridge in einen **eigenen** Prozess laden und die Krypto-Funktionen selbst aufrufen. Mann nutzt die Krypto-Bridge also als „Orakel“ und der Schlüssel muss dafür nie extrahiert werden.

Was ist „Replay-Mint“ — und warum kann man nicht einfach kopieren?

Die internen Krypto-Objekte (Engine, Datenobjekte, Session-Schlüssel) sind **lebende C++-Objekte** auf dem **Heap** (dem Speicherbereich für dynamisch erzeugte Objekte): Sie enthalten Zeiger auf weitere Objekte und

eine **Vtable** (die Tabelle ihrer Methoden). Zieht man sie roh aus einem Speicherabzug und lädt sie in einen anderen Prozess, zeigen all diese Zeiger ins Leere und die Kopie stürzt sofort ab. Das ist die „**Vtable-Objekt-Wall**“.

Der Ausweg ist, die Objekte **nicht zu kopieren, sondern neu zu prägen** (engl. *to mint*): Man spielt die **exakte Initialisierungsreihenfolge** der echten App nach (*replay*) und lässt die White-Box-Engine im **eigenen** Prozess dieselben Objekte frisch erzeugen. Weil sie so im eigenen Adressraum entstehen, sind Vtable und Heap-Zeiger automatisch gültig.

Diese Reihenfolge ist dabei ganz konkret und kein abstraktes Ritual: Zuerst wird das zentrale **Engine-Objekt** der White-Box erzeugt. Dann werden die **eingebetteten statischen Blobs** als deren Datenobjekte importiert (damit „kennt“ die Engine ihr fest eingebautes Schlüssel-Rohmaterial). Erst darauf setzen die **Entpack- und Entschlüsselungs-Schritte** auf. Es ist exakt die Kette, die die App bei jedem Start selbst durchläuft, nur eben im eigenen Prozess nachgezogen. Diese Kombination, Reihenfolge nachspielen **und** Objekte neu prägen, trägt den Namen „**Replay-Mint**“.

Warum funktioniert das in einem fremden Prozess?

Der entscheidende technische Hebel: Die Krypto-Bridge ist in Go geschrieben (via **cgo**, der Brücke, über die Go-Code nativen C-Code einbindet) und bringt eine **eigene Laufzeitumgebung** mit. Diese **initialisiert sich beim Laden der Bibliothek (dlopen) selbständig**. Dieselben Konstruktoren, die beim Start der App laufen, laufen unverändert auch im eigenen Prozess. Deshalb ist das „Nachspielen“ kein blinder Versuch: Nach dem Laden steht die Engine bereits einsatzbereit da, und die statische Initialisierung, auf der alle weiteren Objekte aufbauen, hat sich selbst erledigt.

Ganz geschenkt ist das aber nicht. Das Laden im eigenen Prozess gelingt nur, wenn dort zuerst die **C++-Laufzeitumgebung** bereitliegt, auf der die Go-Bridge aufsetzt. Fehlt diese Abhängigkeit, bricht schon der dlopen-Aufruf ab, noch bevor eine einzige Krypto-Funktion erreichbar ist. Ist diese Voraussetzung erfüllt, greift die Selbst-Initialisierung. Ohne sie wäre der gesamte Orakel-Ansatz aussichtslos.

Wie findet man überhaupt die richtigen Funktionen?

Dass die Runtime sich selbst initialisiert, heißt nur: Die Engine steht **bereit**, sie *rechnet* aber nicht von allein. Um zu ver- und entschlüsseln, muss man die Krypto-Funktionen der White-Box **selbst aufrufen**: Ent-/Verschlüsseln, Schlüssel-Entpacken und Signieren, dazu die internen Erzeuger für Engine und Datenobjekte (die „Replay-Mint“-Kette von oben). Diese Funktionen muss man in der Bibliothek aber erst **finden**. Das vorgehen hierzu ist grob umrissen (und bewusst ohne konkrete Adressen), wie folgt:

- **Abbild greifen:** Zuerst das *entschlüsselte* Laufzeit-Abbild der Bridge sichern (die APK-Kopie ist gepackt und unbrauchbar, siehe §4).
- **Exportierte Funktionen ablesen:** Ein Teil der Funktionen ist *exportiert* und steht in der **dynamischen Symboltabelle** des ELF. Das ist die Liste aller nach außen sichtbaren Funktionen mit Name und Adresse, die der dynamische Linker zwingend braucht. Diese Liste bleibt erhalten, auch wenn die Debug-Symbole entfernt sind. Aus ihr liest man die relative Adresse *direct*. Zur Laufzeit ergibt sich über einen bekannten Anker die Ladebasis und jede weitere Adresse ist dann **Basis + fester Abstand**.
- **Interne Funktionen lokalisieren:** Die *nicht* exportierten Helfer (z. B. die Erzeuger für Engine und Datenobjekte) tauchen im **Live-Trace** als Aufrufer-Adressen auf und werden per Disassembly bestätigt.

Die konkreten Werte sind an eine App-/Bibliotheksversion gebunden und werden hier bewusst nicht genannt.

Damit ist aber erst die halbe Miete geschafft. Eine Krypto-Funktion der White-Box ist keine freistehende Routine, sondern eine **Methode auf einem lebenden Objekt**. Diese Objekte lassen sich nicht einfach aus einem fremden Prozess herüberkopieren (die „Vtable-Objekt-Wall“ von oben). Man braucht also **zweierlei zugleich**: Die *Adressen* der Funktionen (Spur A) und *gültige Objekte*, auf denen man sie aufruft (Spur B, per Replay-Mint frisch geprägt). Abbildung 3 fasst diesen Zusammenhang zusammen:

Vom laufenden Code zum eigenen Orakel

Zwei Voraussetzungen müssen zusammenkommen: die **Adressen** der Krypto-Funktionen (wo) und **gültige Objekte**, auf denen man sie aufruft (worauf).



Kernaussage: Adressen allein genügen nicht — eine Krypto-Funktion ist eine **Methode auf einem lebenden Objekt**. Erst **Spur A** (wo die Funktion liegt) und **Spur B** (ein gültiges Objekt zum Aufrufen) zusammen ergeben ein lauffähiges Orakel. - Bewusst **ohne konkrete Adressen, Symbol- oder Dateinamen** — nur das Prinzip.

Abbildung 3: Vom laufenden Code zum eigenen Orakel.

Vom laufenden Code zum eigenen Orakel: In der App wird die gepackte Bibliothek zur Laufzeit in ein temporäres, entschlüsseltes Abbild entpackt und gesichert. Danach laufen zwei Spuren zusammen: Spur A ermittelt die Adressen der Krypto-Funktionen (exportierte Funktionen über die dynamische Symboltabelle, Anker zur Ladebasis mit Basis-plus-Abstand, interne Helfer per Live-Trace und Disassembly). Spur B löst das Objekt-Problem (die Krypto-Objekte sind lebende C++-Objekte mit Vtable und Heap-Zeigern. Rohes Kopieren scheitert an der Vtable-Objekt-Wall, während Replay-Mint die Objekte frisch im eigenen Prozess prägt, sodass ihre Zeiger gültig sind). Beide Spuren zusammen ergeben ein Orakel im eigenen Prozess, das byte-identisch verifiziert wurde, ohne einen Schlüssel zu extrahieren.

Dieses Verfahren wurde **byte-identisch verifiziert**: Der im eigenen Prozess rekonstruierte Sitzungsschlüssel entschlüsselt eine echte Server-Antwort exakt. Das beweist eine wichtige Eigenschaft: **Die White-Box ist nicht prozessgebunden**. Der Sitzungsschlüssel ergibt sich deterministisch aus in der Bibliothek eingebetteten Daten plus einem pro-Sitzung vom Server gelieferten Bundle. Die **Geräte-Portabilität** folgt aus der Account-Unabhängigkeit dieser eingebetteten Daten, wurde aber nicht über mehrere Geräte hinweg verifiziert (Befund auf einem Gerät).

Was passiert hier methodisch — und was nicht?

Dieser Angriff **bricht die White-Box nicht**: Er greift die mathematische Schlüssel-Verschleierung an keiner Stelle an (kein DCA/DFA, keine Schlüssel-Extraktion). Es handelt sich vielmehr um klassisches „**Code Lifting**“: Die White-Box wird als **Black-Box-Dienstleister im eigenen Prozess** weiterverwendet. Ihre legitime Aufruf-Schnittstelle wird nachgenutzt und exakt so bedient, wie die App es täte. Der Rohschlüssel bleibt die gesamte Zeit im geschützten Inneren der White-Box. Er wird **umgangen, nicht geknackt**. Diese Unterscheidung ist wichtig: Die Härtung der White-Box-Mathematik ist damit nicht widerlegt. Sie ist nur an der falschen Stelle wirksam, da die Aufruf-Schnittstelle im eigenen Prozess reproduzierbar ist.

Läuft das auch in der Praxis?

Verfahren 2 blieb nicht theoretisch: Es wurde als **lauffähiger Proof-of-Concept** umgesetzt. Dabei handelt es sich um einen eigenständigen Client, der sich mit E-Mail und Passwort **anmeldet** und den **eigenen Glukosewert live abruf und entschlüsselt**, ganz ohne die Original-App.

9. Übergreifende Erkenntnisse

- **Doppelte Verschlüsselung (ALE über TLS):** Macht reinen Netzwerk-MITM wirkungslos. Der interessante Punkt liegt dort, wo der Klartext ohnehin entsteht.
- **Laufzeit-Integritätsprüfungen: Schlägt man nicht durch cleverere Byte-Patches am geschützten Code, sondern durch reine Beobachtung ohne Byte-Änderung** an eben diesem Code (Hardware-Breakpoints), während die App an anderer Stelle (per Bytecode-Patch) durchaus modifiziert wird.
- **Mehrschichtiger Schutz erfordert mehrschichtige Antworten:** Nativer Code *und* Java-RASP müssen zusammen betrachtet werden.
- **White-Box heißt nicht automatisch prozessgebunden:** Hier ließ sich die Kette deterministisch offline in einem fremden Prozess reproduzieren. Das ist der eigentlich überraschende Befund. Die Geräte-Portabilität folgt aus der Account-Unabhängigkeit der eingebetteten Blobs, ist aber bisher nur auf einem Gerät belegt.

10. Alternative Wege (Einordnung)

Weg	RE-Aufwand	robust gegen erzwungene ALE	Bewertung
REST-Client (offene Follower-Schnittstelle)	Keiner	Nein (bricht, sobald ALE serverseitig erzwungen wird)	Billigster Weg, solange er hält
On-Device-Hook (Verfahren 1)	Mittel	Ja	Liefert genau das, was die echte App sieht
White-Box-Orakel (Verfahren 2)	Hoch	Ja	Patch-frei, überlebt ALE-Zwang; die sauberste Eigenlösung
Schlüssel-Extraktion (DCA/DFA gegen die White-Box)	Sehr hoch	Ja	Unwirtschaftlich — das Orakel erreicht dasselbe, ohne den Schlüssel zu sehen
Sensor direkt lesen (Jugluco/xDrip u. ä.)	Gering–mittel	Ja (kein Server)	Technisch stark, wenn man den Sensor selbst trägt, entkoppelt aber von LibreLinkUp/LibreView bzw. verlagert den Upload auf Drittanbieter App (Nachteil, wenn der behandelnde Arzt den Follower-Dienst nutzt)

11. Datenschutz

Alle entschlüsselten Daten und alles Schlüsselmaterial sind personenbezogene Gesundheitsdaten und bleiben lokal auf dem eigenen Gerät. In dieser Veröffentlichung kommen weder echte Schlüssel, UUIDs oder E-Mail-Adressen noch build-spezifische Adressen vor.

Dieser Bericht dokumentiert eine persönliche Interoperabilitäts-Lösung und die dabei gewonnenen Erkenntnisse über App-Härtung und White-Box-Kryptografie. Er ist kein Aufruf und keine Anleitung, fremde Daten oder fremde Geräte anzugreifen.

Anhang — Glossar

Die wichtigsten Fachbegriffe dieses Berichts in Kurzform:

Begriff	Bedeutung
TLS	Transportverschlüsselung des „https“ — schützt die Verbindung zum Server.
ALE (Application-Level Encryption)	Verschlüsselung der Nutzdaten auf Anwendungsebene, zusätzlich zu und unabhängig von TLS.
AES-GCM	Symmetrisches Verschlüsselungsverfahren mit eingebautem Echtheits-Siegel (Auth-Tag).
RSA-PSS	Asymmetrisches Signaturverfahren; beweist Echtheit, verschlüsselt selbst nichts.
Nonce	Einmal-Zahl, die dieselbe Nachricht bei jeder Verschlüsselung anders aussehen lässt.
White-Box (Secure Key Box)	Krypto-Verfahren, das mit Schlüsseln rechnet, ohne sie je als lesbaren Wert offenzulegen.
JWT (Zugangs-Ticket)	Vom Server signierter Ausweis, der einen Aufruf einem Konto zuordnet — ohne erneutes Passwort.
RASP (Runtime Application Self-Protection)	Selbstschutz, der Manipulation/Debugger zur Laufzeit erkennt und die App gezielt abstürzen lässt.
cgo	Die Brücke, über die in Go geschriebener Code nativen C-Code einbindet.
Vtable	Die Methodentabelle eines C++-Objekts; enthält Zeiger auf dessen Funktionen.
Heap	Der Speicherbereich für dynamisch erzeugte Objekte; ihre Adressen gelten nur im eigenen Prozess.
ELF	Das Dateiformat ausführbarer Programme und Bibliotheken unter Linux/Android.
Dynamische Symboltabelle	Die im ELF verbliebene Liste nach außen sichtbarer Funktionen (Name + Adresse), die der Linker braucht.
AAPCS64	Die ARM64-Aufrufkonvention: welche Funktionsargumente in welchen CPU-Registern stehen.
Hardware-Breakpoint	CPU-gestützter Halt an einer Adresse — beobachtet Code, ohne ihn zu verändern.
Code Lifting	Nachnutzen einer legitimen Krypto-Schnittstelle im eigenen Prozess, ohne den Schlüssel zu extrahieren.